



Globally convergent exact and inexact parametric algorithms for solving large-scale mixed-integer fractional programs and applications in process systems engineering

Zhixia Zhong, Fengqi You*

Department of Chemical and Biological Engineering, Northwestern University, Evanston, IL 60201, USA



ARTICLE INFO

Article history:

Received 15 May 2013

Received in revised form 23 October 2013

Accepted 24 October 2013

Available online 2 November 2013

Keywords:

Global optimization

Parametric approach

Inexact algorithms

Process scheduling

Mixed-integer fractional programming

ABSTRACT

This paper is concerned with the parametric algorithms for solving large-scale mixed-integer linear and nonlinear fractional programming problems, as well as their application in process systems engineering. By developing an equivalent parametric formulation of the general mixed-integer fractional program (MIFP), we propose four exact parametric algorithms based on the root-finding methods, including bisection method, Newton's method, secant method and false position method, respectively, for the global optimization of MIFPs. We also propose an inexact parametric algorithm that can potentially outperform the exact parametric algorithms for some types of MIFPs. Extensive computational studies are performed to demonstrate the efficiency of these parametric algorithms and to compare them with some general-purpose mixed-integer nonlinear programming methods. The applications of the proposed algorithms are illustrated through two case studies on process scheduling. Computational results show that the proposed exact and inexact parametric algorithms are more computationally efficient than several general-purpose solvers for solving MIFPs.

© 2013 Elsevier Ltd. All rights reserved.

1. Introduction

A mixed-integer fractional programming (MIFP) problem has an objective as a ratio of two functions, and includes both discrete and continuous variables. A general form of MIFP can be stated as the following problem (P):

$$\max \left\{ Q(x) = \frac{N(x)}{D(x)} \mid x \in S \right\} \quad (P)$$

where the variables x contain both continuous and discrete variables, the feasible region S is nonempty, compact, bounded and the denominator function $D(x)$ is always positive in S , i.e. $D(x) > 0$ for $x \in S$ (Frenk & Schaible, 2009). The numerator function $N(x)$ and the denominator function $D(x)$ can be linear or nonlinear.

1.1. MIFP applications in process systems engineering

MIFP problems arise from a variety of real world applications (Schaible, 1981). Major MIFP applications in process systems engineering can be generally categorized into three types.

The first one is to optimize the productivity of a process system, which can usually be measured by the performance per unit of time or the process output per input. These include, but are not limited to

overall cost or profit over the makespan, the resource generation or consumption rate, and the product yield per raw material or utility consumption. Production scheduling problems can be formulated as a mixed-integer linear fractional programming (MILFP) problem by optimizing the productivity subject to the mixed-integer linear constraints (Capón-García, Bojarski, Espuña, & Puigjaner, 2010; Shah, Pantelides, & Sargent, 1993). Cyclic process operations problems (Chu & You, 2012, 2013; Pinto & Grossmann, 1994) might involve the tradeoffs between inventory cost and fixed cost in the objective function that would lead to a mixed-integer quadratic fractional program (MIQFP).

Another important MIFP application is optimization for sustainability. Although overall environmental impact is usually used as the objective function for process optimization problems, environmental impact per functional unit could sometimes be a more appropriate objective function from the life cycle assessment perspective, especially for the problems focusing on the performance of products, rather than that of the entire process. Recent MIFP applications in this area include environmental-conscious sustainable scheduling of batch processes (Capón-García et al., 2010; Yue & You, 2013) and sustainable design of supply chains (Yue, Kim, & You, 2013).

The third MIFP application field is on the optimization for return rate, such as return on investment, return on cost and return on risk. The return rate is usually modeled by profit dividing the capital, revenue, asset or risk. One example is that capacity planning problems can be formulated as an MILFP problem to simultaneously

* Corresponding author. Tel.: +1 847 467 2943; fax: +1 847 491 3728.
E-mail address: you@northwestern.edu (F. You).

optimize the decisions of capacity investment, inventory management, and production planning by maximizing the return over operating assets (Bradley & Arntzen, 1999). Portfolio selection problems can use financial return over risk as the objective function. Supply chain design problems can also be formulated as an MILFP by optimizing the total return over capital investment (Uddin & Sano, 2010) or market share capture per unit cost (Hua, Cheng, & Wang, 2011).

1.2. MIFP theory and algorithms

The nonlinear relaxation of an MIFP (by relaxing the integer variables to continuous variables with corresponding upper and lower bounds) is a continuous fractional program. Due to the ratio term in the objective function, fractional programs are in general non-convex nonlinear optimization problems (Bazaraa, Sherali, & Shetty, 2006). Various solution methods for continuous fractional programs have been proposed (Schaible, 1981). Charnes and Cooper (1962) proposed an exact linear programming reformulation of the continuous linear fractional program. An alternative approach is to solve the associated dual problem of the variable transformed fractional program that could be efficient for a class of quadratic fractional programs (Schaible, 1976). Another popular solution method was first introduced by Martos and Andrew Whinston (1964) and Jagannathan (1966) and they named it as the so-called “parametric” approach. The main idea of this parametric approach is to solve an equivalent parametric problem of the fractional program. Dinkelbach (1967) extended the parametric approach to solve the continuous, nonlinear fractional programs using the Newton’s method. There are also several variants based on this parametric approach but using different methods for updating the parameters (Schaible & Ibaraki, 1983).

Although earlier work mainly focuses on fractional programming problems with only continuous variables, solution methods for MIFP problems drew significant attentions recently. MIFP problem is a special class of mixed-integer nonlinear programming (MINLP) problem. Thus, MIFP problems can be solved with the general-purpose MINLP algorithms (Floudas, 1999), such as the generalized Benders decomposition (Geoffrion, 1972), branch-and-bound method (Gupta & Ravindran, 1985), the outer-approximation method (Duran & Grossmann, 1986; Viswanathan & Grossmann, 1990), the branch-and-cut method (Quesada & Grossmann, 1992), the extended cutting plane method (Westerlund & Pettersson, 1995), the branch-and-reduce method (Tawarmalani & Sahinidis, 2005), the outer-approximation-based global optimization method (Kesavan & Barton, 2000a, 2000b; Kesavan, Allgor, Gatzke, & Barton, 2004), just to name a few. However, due to the nature of nonconvexity and the presence of integer variables, solving large-scale MIFP problems directly using the general-purpose MINLP methods might be computationally intractable. Moreover, local MINLP solvers such as DICOPT (outer-approximation algorithm) and SBB (simple branch-and-bound algorithm) might not guarantee the global optimality of the solution and may lead to suboptimal solutions if they are used for solving the MIFP problems. As will be shown through the computational results in Sections 3 and 4, the global optimizer BARON 11.3 might not be able to return good feasible solutions of large-scale MIFP problems within reasonable computational time limit. Therefore, there is a need of developing efficient and tailored global optimization algorithms for solving large-scale, non-convex MIFP problems.

Most tailored MIFP solution algorithms in the literature focus on the special case of MILFP problems. Anzai (1973) investigated the properties of integer fractional programming problems. Granot and Granot (1977) developed valid cutting planes based on the Charnes-Cooper transformation for solving MILFP problems. A global

optimization approach based on the branch-and-bound method and variable transformation to solve 0–1 fractional programs was proposed by Li (1994), which was extended by Wu (1997) to derive stronger cuts for the 0–1 fractional programs. Chang (2002) proposed an approximate approach to solving posynomial fractional programming problems by deriving the linear programming relaxation of the problem based on piecewise linearization techniques. Yue, Guillén-Gosálbez, and You (2013) proposed an exact mixed-integer linear programming (MILP) reformulation for MILFP problems, although it cannot be applied to mixed-integer nonlinear fractional programs. An algorithm based on the parametric approach was proposed to solve integer linear fractional programming problems by Ishii, Ibaraki, and Mine (1977). Pochet and Warichet (2008) and You, Castro, and Grossmann (2009) showed that the parametric approach is very efficient for solving MILFP models for cyclic scheduling, but their studies were not extended to the general mixed-integer nonlinear fractional programs that would be addressed in this paper.

1.3. Outline of this paper

The goal of this paper is to propose novel and efficient algorithms based on the parametric approach for solving MIFP problems, and to illustrate their effectiveness of solving process scheduling problems. We first show that the parametric approach for continuous fractional programs is applicable to the mixed-integer linear and nonlinear fractional programs. To solve the resulting parametric problem, which is equivalent to the original MIFP problem, we consider four one-dimension root-finding algorithms, including the bisection method, the Newton’s method, the secant method and the false position method. These are all globally convergent exact methods with at least linear convergence rates. We further propose a novel, inexact parametric algorithm based on the Newton’s method for solving the equivalent parametric problem. We show that this new algorithm has a linear convergence rate, but it could be much more computationally efficient in each iteration than the exact parametric algorithms. Thus, the inexact parametric algorithm can potentially require shorter total computational times than the exact parametric algorithms for some types of MIFPs.

We conduct extensive computational studies based on the randomly generated instances, in order to illustrate the efficiency of the proposed parametric algorithms for solving large-scale MILFP and MIQFP problems, and also to compare their performance with three general-purpose MINLP methods. Two applications of these algorithms in process operations are presented at the end of this paper.

The novel contributions of this paper are summarized below:

- Four efficient exact parametric algorithms for solving large-scale MIFP problems based on the application of existing root-finding algorithms;
- An inexact parametric algorithm based on the Newton’s method for solving special structured MIFP problems, and the theoretical investigation of its convergence properties;
- Detailed comparison between the proposed parametric approaches and the general-purpose MINLP algorithms (DICOPT, SBB, BARON 11.3) for solving large-scale MILFP and MIQFP problems;

2. Parametric algorithms for solving MIFP problems

2.1. Equivalent parametric formulation and its properties

Considering the following parametric problem (P_q):

$$F(q) = \max \{ N(x) - q \cdot D(x) | x \in S \} \quad \text{for } q \in \mathbb{R}^1 \quad (P_q)$$

where x contains continuous variables and integer variables, S is nonempty, compact and bounded, $D(x) > 0$, and q is a parameter.

This parametric problem (P_q) has some special properties that can be utilized for solving the original fractional program (P) . Specifically, we show through Lemmas 1–5 in Appendix A that the function $F(q)$ is convex, strictly monotonic decreasing, continuous and has bounded subgradients for $q \in \mathbb{R}^1$, and that $F(q) = 0$ has a unique solution q^* which is exactly the global optimal objective value of the problem (P) . Thus, we have the following proposition for the equivalence between the parametric problem and the fractional program.

Proposition (equivalence). $q^* = N(x^*)/D(x^*) = \max \{N(x)/D(x) | x \in S\}$ if and only if $F(q^*) = F(q^*, x^*) = \max \{N(x) - q^* D(x) | x \in S\} = 0$, i.e. the variable x^* is a global optimal solution to the fractional programming problem (P) if and only if x^* is a global optimal solution to the parametric problem (P_q) with the parameter q^* such that $F(q^*) = 0$ (You et al., 2009).

We note that the equivalence proposition only requires that the denominator is always positive, i.e. $D(x) > 0$. The proposition holds true when the problem contains both continuous and discrete variables, no matter whether the functions $N(x)$ and $D(x)$ are linear or nonlinear. Though there are no assumptions on the convexity of the two functions $N(x)$ and $D(x)$, the equivalence is only valid based on the global optimal solutions of both problems. In other words, the equivalence proposition might be invalid for local optimal solutions of the problems.

Because the ratio function in the objective is transformed into the difference, the parametric problem (P_q) has fewer nonlinearities than the MIFP problem (P) , and might be easier to solve. For example, if (P) is an MILFP problem, (P_q) is a (parametric) mixed-integer linear program (MILP); if (P) is an MIQFP problem with a convex $N(x)$ and a concave $D(x)$, (P_q) is a convex mixed-integer quadratic program (MIQP). MILP and convex MIQP problems can be solved very efficiently using the state-of-the-art branch-and-cut methods implemented in solvers like CPLEX. Thus, the parametric approach might potentially be much more computationally efficient than the general-purpose MINLP methods for solving these types of MIFP problems.

As will be presented in the next section, the proposed parametric approaches do not solve the parametric mixed-integer programming problems (Bank, Guddat, Klatte, Kummer, & Tammer, 1983; Gal, 1995; Mitsos & Barton, 2009; Murty, 1980; Pistikopoulos, Georgiadis, & Dua, 2007). The main idea of the parametric approach is using root-finding methods to solve the nonlinear equation for the parametric form of the MIFP problem (P_q) .

2.2. The algorithms for solving parametric problem (P_q)

Based on the properties of the parametric problem (P_q) , it is easy to see that

$$F(q) > 0 \quad \text{iff} \quad q < q^*$$

$$F(q) = 0 \quad \text{iff} \quad q = q^*$$

$$F(q) < 0 \quad \text{iff} \quad q > q^*$$

Therefore, the solution of the MIFP problem (P) ends up with finding the root of equation $F(q) = \max \{N(x) - q \cdot D(x) | x \in S\} = 0$. Although $F(q)$ does not have a close-form analytical expression, there are a number of root-finding algorithms for solving nonlinear equations, such as the bisection method, Newton's method, secant method, false position method, interpolation method and inverse interpolation method, as well as their combinations. In principle,

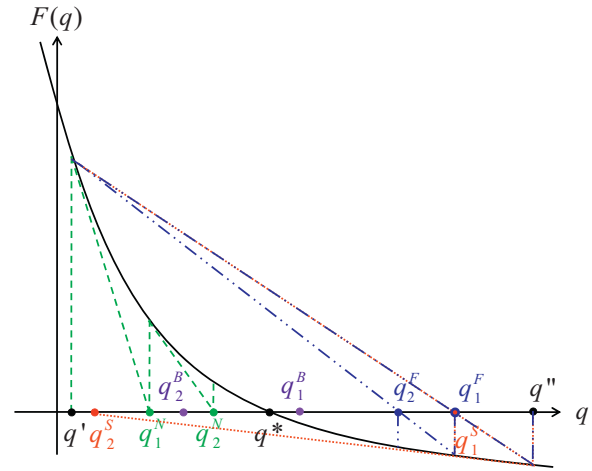


Fig. 1. Iteration procedures of solving the equivalent parametric problem by using the bisection method (q^B), the exact Newton's method (q^N), the secant method (q^S) and the false position method (q^F). q' and q'' are the starting points, and q^* is the optimal solution.

all these methods are applicable for solving the parametric problem (P_q) . In this section, we apply four well-known root-finding algorithms to solve the equivalent parametric form of MIFP. The iteration procedures of these methods are shown in Fig. 1.

We only introduce the Newton's method in details in the main text, because it will be shown to be the most efficient one through the computational studies in Sections 3 and 4. The secant, bisection and false point methods are presented in Appendix B. We note that the Newton and secant methods both have superlinear convergence rate, while the bisection and false position methods have only linear convergence rate.

2.3. Exact parametric algorithm based on Newton's method

The application of Newton's method to solve linear continuous fractional programs was first proposed by Isbell and Marlow (1956) and then generalized to nonlinear fractional programs by Ritter (1962) and Dinkelbach (1967). Thus, the Newton's method for fractional programs is usually called the Dinkelbach's algorithm.

In Newton's method, q_{n+1} is defined by, $q_{n+1} = q_n - (F(q_n)/F'(q_n))$. As shown in Lemma 4, we can use the approximated subgradient at point q_n to estimate the derivative, $F'(q_n) = dF(q_n)/dq_n \approx -D(x_n^*)$, which is the negative value of the denominator evaluated at x_n^* , a global optimal solution of $\max \{N(x) - q_n \cdot D(x) | x \in S\}$.

Thus, we have $q_{n+1} = q_n - \frac{F(q_n)}{-D(x_n^*)} = q_n + \frac{N(x_n^*) - q_n \cdot D(x_n^*)}{D(x_n^*)} = \frac{N(x_n^*)}{D(x_n^*)}$.

The full procedure of the exact Newton's method for solving (P_q) is as follows:

Step 1. Choose arbitrary $x_0 \in S$ and set $q_1 = N(x_0)/D(x_0)$ or simply set $q_1 = 0$. Set $n = 1$.

Step 2. Solve $F(q_n) = \max \{N(x) - q_n \cdot D(x) | x \in S\}$. Denote the optimal solution as x_n^* .

Step 3. If $|F(q_n)| < \delta$ (optimality tolerance), stop and output x_n^* as the optimal solution and q_n as optimal objective. If $|F(q_n)| \geq \delta$, let $q_{n+1} = N(x_n^*)/D(x_n^*)$, update n with $n + 1$ and update q_n with q_{n+1} . Go to Step 2.

In this method, the function is approximated by its tangent line and one computes the q -intercept of this tangent line. The method can iterate and converge to q^* as shown in Fig. 1.

We note that this is an exact Newton's method that requires each parametric subproblem to be solved to the global optimum.

This algorithm converges superlinearly at a rate of $[1 - D(x^*)/D(x')]$ as described in Lemmas 6–8 in Appendix C.

You et al. (2009), which is based on Dinkelbach (1967), presented a method for solving the mixed-integer linear fractional programming problems that is exactly the same as this exact Newton's parametric method.

2.4. Inexact parametric algorithm based on Newton's method

The parametric algorithms presented in the previous section are all globally convergent exact methods. In each step of those algorithms, the parametric subproblem needs to be solved to the global optimality, i.e. 0% optimality gap. Mixed-integer programming problems are usually NP-hard and the solution to 0% optimality gap might require significant computational effort when the problem size is large. As an example, for certain MILP problems, the computational time required to solve the problem to 10% gap could be much shorter than that for closing the optimality gap from 10% to 0%.

In this section, we present an inexact solution approach based on the Newton's method for solving MIFPs, of which both the numerator and denominator functions are non-negative, i.e. $N(x) \geq 0$ and $D(x) > 0$. The main idea is that we start the iterations with the initial value of $q = 0$, and apply the same procedure as the Newton's method, except that the parametric subproblem in each iteration does not need to be solved to the global optimality, i.e. 0% gap. Instead, we show that the algorithm will still converge to q^* , as long as that we set the initial value of $q = 0$ and that each subproblem is solved to a pre-defined relative optimality criterion, which is less than 100%. Here we consider the same definition of relative optimality criterion as used by GAMS, i.e. $(|BP - BF|)/|BP| < \varepsilon$, where BP is the best possible solution, BF is the best solution found, and ε is the relative optimality criterion (Rosenthal, 2011).

The full procedure of the inexact parametric algorithm based on the Newton's method for solving (P_q) is as follows:

Step 1. Set $q_1 = 0$ and initialize n by setting $n = 1$.

Step 2. Solve the problem $F(q_n) = \max \{N(x) - q_n \cdot D(x) | x \in S\}$ to a relative optimality gap ε , where $\varepsilon < 100\%$. Denote the near-optimal solution as x_n^* .

Step 3. If $|F(q_n)| < \delta$ (tolerance), stop and output x_n^* as the optimal solution and q_n as optimal objective. If $|F(q_n)| \geq \delta$, let $q_{n+1} = N(x_n^*)/D(x_n^*)$, update n with $n + 1$ and update q_n with q_{n+1} . Go to Step 2.

We can see that the main difference between this algorithm and the one presented in Section 2.3 is the Step 2, where each subproblem does not need to be solved to global optimum. This can potentially reduce the computational time required for each iteration, although the number of iterations might increase. As shown in Appendix D through Lemmas 9–11 and Remarks 1–2, this inexact algorithm converges linearly with a rate of $[1 - D(x^*)/(\eta \cdot D(\bar{x}_k))]$, where $\bar{x}_k$ is the near-optimal solution with a relative optimality gap ε obtained in iteration k and $\eta = 1/(1 - \varepsilon)$.

As will be shown through two case studies in Section 4, this inexact parametric algorithm can sometime outperform the exact algorithms for some process scheduling applications.

3. Comparison of computational efficiency of solution algorithms

In order to test the computational efficiency of the proposed solution algorithms for solving large-scale MIFP problems, we conduct computational experiments based on two testing problems.

The MILFP testing problem (TP1) is presented as follows:

$$\begin{aligned} \max \quad & \frac{A_0 + \sum_{i \in I} A1_i x_i + \sum_{j \in J} A2_j y_j}{B_0 + \sum_{i \in I} B1_i x_i + \sum_{j \in J} B2_j y_j} \\ \text{s.t.} \quad & \sum_{i \in I} C1_{ik} x_i + \sum_{j \in J} C2_{jk} y_j \leq D_k, \quad \forall k \in K \\ & B_0 + \sum_{i \in I} B1_i x_i + \sum_{j \in J} B2_j y_j \geq 0.001 \\ & x^L \leq x_i \leq x^U, \quad \forall i \in I \text{ and } y_j \in \{0, 1\}, \quad \forall j \in J \end{aligned} \quad (\text{TP1})$$

where all the constraints are linear and the objective function is the ratio of two linear functions.

The MIQFP testing problem (TP2) is given below:

$$\begin{aligned} \max \quad & \frac{A_0 + \sum_{i \in I} A1_i x_i + \sum_{j \in J} A2_j y_j - \sum_{i \in I} A3_i x_i^2}{B_0 + \sum_{i \in I} B1_i x_i + \sum_{j \in J} B2_j y_j + \sum_{i \in I} B3_i x_i^2} \\ \text{s.t.} \quad & \sum_{i \in I} C1_{ik} x_i + \sum_{j \in J} C2_{jk} y_j \leq D_k, \quad \forall k \in K \\ & B_0 + \sum_{i \in I} B1_i x_i + \sum_{j \in J} B2_j y_j \geq 0.001 \\ & x^L \leq x_i \leq x^U, \quad \forall i \in I \text{ and } y_j \in \{0, 1\}, \quad \forall j \in J \end{aligned} \quad (\text{TP2})$$

where all the constraints are linear and the objective function has a concave quadratic function in the numerator and a convex quadratic function in the denominator.

We note that the second constraint in both testing problems is to ensure that the denominator is positive. Both problems consists of $|I|$ continuous variables, $|J|$ binary variables, and $|K| + 1$ constraints. The values of $|I|$, $|J|$ and $|K|$ range from 100 to 2000. All parameters are generated randomly. Details about the coefficients are provided in the supporting information.

For each testing problem, we solve 45 large-scale, randomly generated instances to demonstrate the efficiency of the solution algorithms, including the exact Newton's method, bisection method, secant method, false position method and the general MINLP solution methods via SBB solver (simple branch-and-bound algorithm), DICOPT solver (outer-approximation algorithm) and the global optimizer BARON 11.3 (Tawarmalani & Sahinidis, 2005). A tailored MILFP solution approach, the reformulation linearization method (Yue et al., 2013b), is also used to solve the MILFP problem (TP1) for comparison. We note that this reformulation method cannot be used to solve mixed-integer nonlinear fractional programs like the MIQFP problem (TP2). Default initialization options by GAMS are used for the MINLP solvers. All the computational experiments are carried out on a computer with Intel® Core™ 2 Due CPU P8600@2.4 GHz 1.58 GHz and 2.98 GB RAM. Only one core is used for all computation. All models and solution procedures are coded in GAMS 23.9. The MILP and MIQP problems in the parametric algorithms are solved using CPLEX 12, which was called in cold-start at each iteration of the parametric algorithm. The relative optimality tolerance of the MILP/MIQP solvers are set to 0 and the absolute optimality tolerance for the root-finding algorithms are set as $\delta = 10^{-3}$. The maximum solution time for each instance is limited to 2 h or 7200 CPU s. The detailed computational results for all the instances are listed in the supporting information.

Fig. 2 shows the performance profiles of solving the MILFP testing problems (TP1) using the aforementioned algorithms, based on the performance analysis and benchmarking methods of optimization algorithms proposed by Dolan and Moré (2002) and Bussieck et al. (2007). The x-axis is the maximum computational time needed for solving the problem and the y-axis is the number of instances. If a solution method has a performance profile toward the upper left corner, it implies that this method can solve more problems within shorter time, i.e. better computational performance. We can see from Fig. 2 that the performances of the parametric algorithms

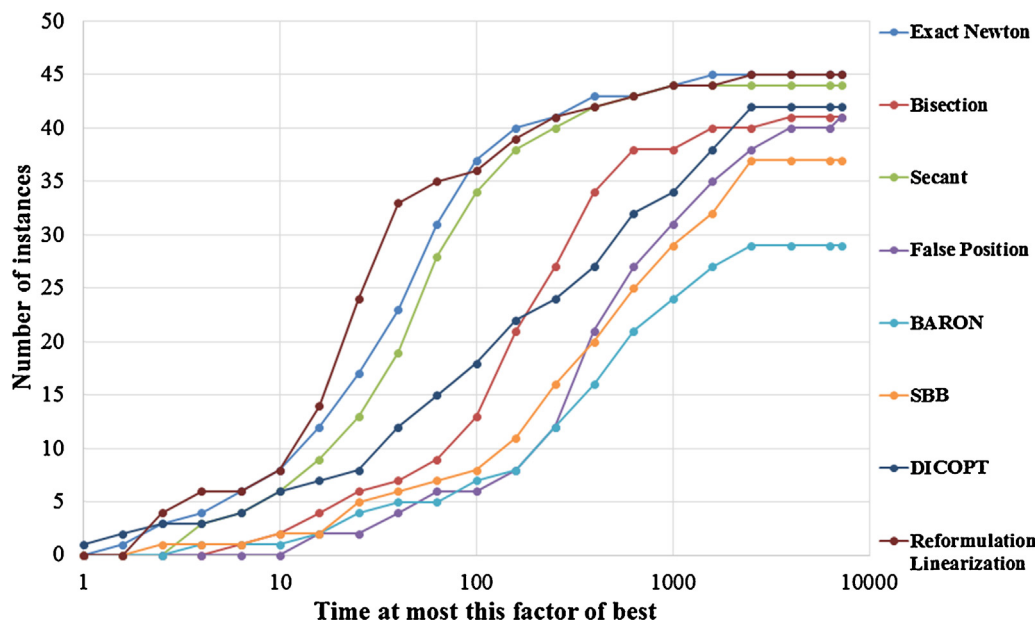


Fig. 2. Performance profile for the solution of the 45 instances of MILFP testing problem (TP1).

and reformulation-linearization method are in general better than those tested MINLP solvers in solving the MILFP problems. Among the parametric algorithms, Newton's method and secant method, which both converge quadratically, have better performance than the bisection method and the false position method with linear convergence rates. Generally speaking, among the four tested parametric algorithms, both exact Newton's method and the secant method are good choices in solving large-scale MILFP problems. Reformulation-linearization method has comparable performance with the exact Newton's method and the secant method for solving MILFP problems.

The performance profiles of solving the MIQFP testing problems (TP2) are given in Fig. 3. We can clearly see that the exact Newton's method is the "champion" among all the solution methods considered in this test, similar to the MILFP cases. It is then followed by the secant method. However, the secant method needs two starting points with opposite signs in $F(q)$ that might be less convenient to implement than the exact Newton's method. For this MIQFP cases, the performance of the general-purpose MINLP solvers SBB exceeds those of another MINLP solver DICOPT and the global optimizer BARON, as well as the bisection method and the false position method. In general, exact Newton's method which combines the parametric approach and Newton's solution method has obvious advantages in solving MIFP, including MILFP and mixed-integer nonlinear fractional programs such as MIQFP problems, due to its high computational efficiency and guarantee of finding global optimal solutions.

These randomly generated instances, due to their relatively dense coefficient matrices, might not fully capture the properties of MIFP problems from practice, but they might shed light on the computational efficiency of the algorithms. In Section 4, we will present two real world application problems of MIFP and the corresponding computational results to further compare the proposed algorithms with the general-purpose MINLP methods.

4. Case studies

In this section, we consider two case studies to illustrate the application of the proposed algorithms for solving process systems engineering problems. Through the two examples, we will compare the computational efficiency of the exact parametric

algorithms, including exact Newton's method, bisection method, secant method, and false position method, the inexact parametric algorithm, and the general-purpose MINLP algorithms via SBB solver (simple branch-and-bound algorithm), DICOPT solver (outer-approximation algorithm) and the global optimizer BARON 11.3. The case studies will also demonstrate the economic benefits and significances of the proposed, more efficient solution algorithms for large-scale MIFP problems. All the computational experiments are carried out on a computer with Intel® Core™ 2 Due CPU P8600@ 2.4 GHz 1.58 GHz and 2.98 GB RAM. Only one core is used for all computation. All models and solution procedures are coded in GAMS 23.9. Default initialization options by GAMS are used for the MINLP solvers. The MILP and convex MIQP problems are solved using CPLEX 12, which was called in cold-start at each iteration of the parametric algorithm. The absolute optimality tolerance for the parametric algorithms are set as $\delta = 10^{-3}$.

4.1. Scheduling problem (mixed-integer linear fractional program)

The first example is a multipurpose batch plant scheduling problem originally from The Dow Chemical Company (Chu, Wassick, & You, 2013; Wassick et al., 2012). As the goal of this section is to compare the computational efficiency of the methods in solving real world MIFP problems rather than the problems themselves, the detailed description of this scheduling problem and the model formulation are presented in the supporting information. The model formulation ends up with a mixed-integer linear fractional program which maximizes the productivity that is represented by the total profit over the makespan. The main tradeoff in this problem is that more time points $|t|$ would potentially lead to better objective value, i.e. higher productivity, but it could also lead to larger problem with more variable and constraints that might be computationally intractable. We therefore tested the problem under different values of $|t|$, using the proposed parametric algorithms (MILP solver CPLEX), reformulation linearization method, MINLP solvers SBB, DICOPT and global optimizer BARON. Table 1 lists the results obtained.

As we can see, when the total time points $|t|$ increases from 6 to 10, the objective, productivity, increases from \$27.00/h to \$39.91/h. Meanwhile, the problem size, as well as the computational

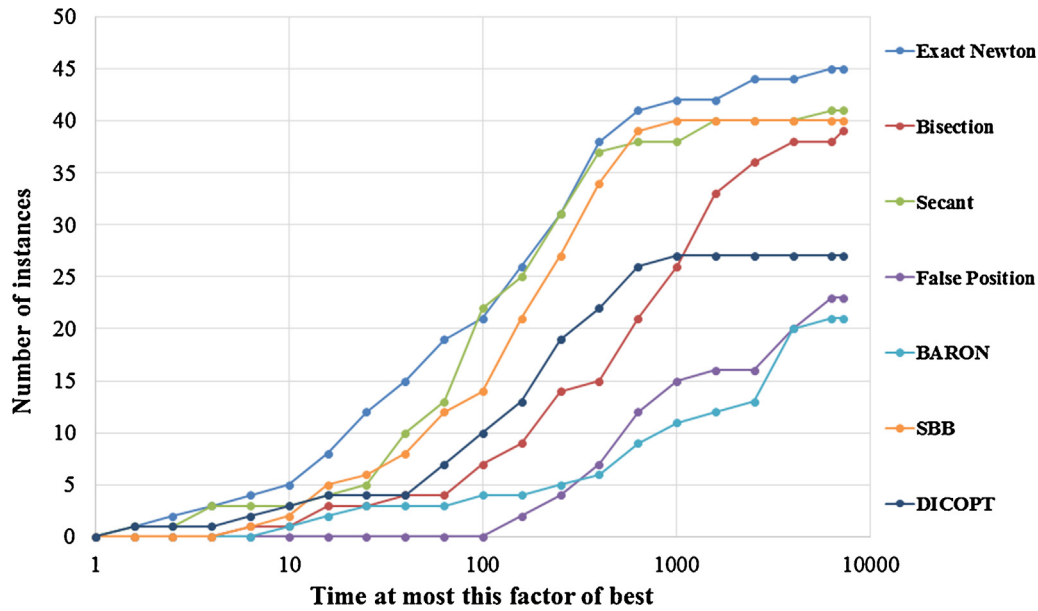


Fig. 3. Performance profile for the solution of the 45 instances of MIQFP testing problem (TP2).

complexity, increases. The proposed parametric algorithms can solve all of the instances listed in the table, except for the false position method in the largest instance. The inexact Newton's method is the most efficient method in solving instance 2, 4 and 5, and the reformulation-linearization is the most efficient method in solving instance 1 and 3. The tested general MINLP solvers are much less efficient than the proposed parametric algorithms for all instances. The MINLP solver SBB only obtained solution in Instance 1 and global optimizer BARON cannot return any feasible solution within the 2 h limit for all instances. The solver DICOPT obtained solutions in instance 1–4, but it is a local MINLP solver and cannot guarantee the solutions' global optimality.

We specifically compare the exact Newton's method with the inexact parametric algorithm proposed in Section 2.4. The results are shown in Table 1 and illustrated in Fig. 4. By using the inexact parametric algorithm, each MILP subproblem only needs to

be solved to a pre-defined relative optimality criterion between 0 and 100%. We implemented this inexact algorithm to solve the scheduling problem with different values of the relative optimality gap. We note that the case of 0% gap corresponds to the exact Newton's method, while the rest cases with 1%, 10% and 90% gaps are for the inexact parametric algorithm. The results show that the inexact algorithm is more efficient than the exact Newton's method for solving this scheduling problem. For all the instances, the inexact algorithm returns the same optimal solution as the exact one, but only requires around 50–70% of the CPU times of the exact Newton's algorithm. The time-saving is more significant for large instance with 10 time points. From the number of iterations listed in Table 1, it is interesting to see that the inexact algorithms with 1% and 10% gap require the same number of iterations as the exact algorithm, i.e. 0% gap. Because solving each MILP subproblem to 1% or 10% gap requires less computational

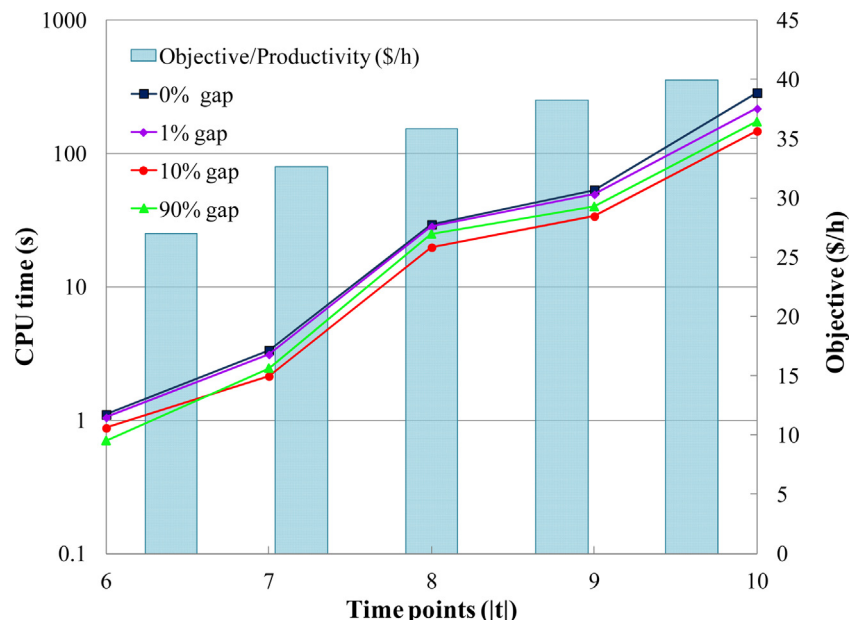


Fig. 4. The comparison of the computational results and efficiency of using inexact Newton's method in the scheduling problem at different relative optimality gaps (optcr).

Table 1
Model statistics and computational results of the scheduling problem (case study 1).

Instance	t	Discrete var.	Continuous var.	Constraint	Obj. (\$/h)	CPU(s)	Solver
1	6	110	702	1562	27.00	1.105 (4 iter.)	Exact Newton's
					27.00	1.047 (4 iter.)	Inexact Newton's (Gap 1%)
					27.00	0.875 (4 iter.)	Inexact Newton's (Gap 10%)
					27.00	0.703 (4 iter.)	Inexact Newton's (Gap 90%)
					27.00	5.955 (23 iter.)	Bisection
					27.00	3.375 (5 iter.)	Secant
					27.00	5.531 (11 iter.)	False position
					27.00	0.375	Reformulation Linearization
					23.69–38.66	7200 ^b	BARON 11.3
					27.00	63.16	SBB
					25.05 ^a	0.562	DICOPT
					32.59	3.344 (3 iter.)	Exact Newton's
					32.59	3.109 (3 iter.)	Inexact Newton's (Gap 1%)
					32.59	2.141 (3 iter.)	Inexact Newton's (Gap 10%)
2	7	132	814	1822	32.59	2.453 (4 iter.)	Inexact Newton's (Gap 90%)
					32.59	13.38 (24 iter.)	Bisection
					32.59	3.939 (5 iter.)	Secant
					32.59	10.06 (7 iter.)	False position
					32.59	3.032	Reformulation Linearization
					19.63–175.8	7200 ^b	BARON 11.3
					32.59–40.00	7200 ^b	SBB
					25.05 ^a	9.704	DICOPT
					35.81	29.34 (4 iter.)	Exact Newton's
					35.81	28.30 (4 iter.)	Inexact Newton's (Gap 1%)
					35.81	19.80 (4 iter.)	Inexact Newton's (Gap 10%)
					35.81	24.84 (5 iter.)	Inexact Newton's (Gap 90%)
					35.81	242.3 (28 iter.)	Bisection
					35.81	58.41 (7 iter.)	Secant
3	8	154	926	2082	35.81	93.44 (10 iter.)	False position
					35.81	19.09	Reformulation Linearization
					– ^c	7200 ^c	BARON 11.3
					34.45–120.9	7200 ^b	SBB
					26.48 ^a	114.0	DICOPT
					38.23	53.39 (3 iter.)	Exact Newton's
					38.23	49.31 (3 iter.)	Inexact Newton's (Gap 1%)
					38.23	33.86 (3 iter.)	Inexact Newton's (Gap 10%)
					38.23	39.95 (3 iter.)	Inexact Newton's (Gap 90%)
					38.23	571.6 (23 iter.)	Bisection
					38.23	713.3 (7 iter.)	Secant
					38.23	568.3 (10 iter.)	False position
					38.23	78.47	Reformulation Linearization
					– ^c	7200 ^c	BARON 11.3
4	9	176	1038	2342	35.42–450.7	7200 ^b	SBB
					26.48 ^a	1781	DICOPT
					39.91	285.7 (3 iter.)	Exact Newton's
					39.91	217.3 (3 iter.)	Inexact Newton's (Gap 1%)
					39.91	147.5 (3 iter.)	Inexact Newton's (Gap 10%)
					39.91	173.0 (4 iter.)	Inexact Newton's (Gap 90%)
					39.91	3627 (29 iter.)	Bisection
					39.91	5103 (6 iter.)	Secant
					– ^c	7200 ^c	False position
					39.91	432.1	Reformulation Linearization
					– ^c	7200 ^c	BARON 11.3
					36.23–793.1	7200 ^b	SBB
					– ^c	7200 ^c	DICOPT
					39.91	285.7 (3 iter.)	Exact Newton's
5	10	198	1150	2602	39.91	217.3 (3 iter.)	Inexact Newton's (Gap 1%)
					39.91	147.5 (3 iter.)	Inexact Newton's (Gap 10%)
					39.91	173.0 (4 iter.)	Inexact Newton's (Gap 90%)
					39.91	3627 (29 iter.)	Bisection
					39.91	5103 (6 iter.)	Secant
					– ^c	7200 ^c	False position
					39.91	432.1	Reformulation Linearization
					– ^c	7200 ^c	BARON 11.3
					36.23–793.1	7200 ^b	SBB
					– ^c	7200 ^c	DICOPT
					39.91	285.7 (3 iter.)	Exact Newton's
					39.91	217.3 (3 iter.)	Inexact Newton's (Gap 1%)
					39.91	147.5 (3 iter.)	Inexact Newton's (Gap 10%)
					39.91	173.0 (4 iter.)	Inexact Newton's (Gap 90%)
					39.91	3627 (29 iter.)	Bisection
					39.91	5103 (6 iter.)	Secant
					– ^c	7200 ^c	False position
					39.91	432.1	Reformulation Linearization
					– ^c	7200 ^c	BARON 11.3
					36.23–793.1	7200 ^b	SBB
					– ^c	7200 ^c	DICOPT

^a Suboptimal solutions.
^b Lower and upper bounds obtained after 2 h (7200 s).
^c No solution returned after 2 h (7200 s), i.e. preprocessing took more than 2 h.

times than solving it to 0%, it is not surprising that the inexact algorithms with 1% and 10% have better computational performance than the exact algorithm. The inexact approach with 90% gap, on the other hand, usually requires another iteration to reach the tolerance, so it might not be as efficient as similar algorithms with 1% or 10% gap. The high efficiency of the inexact algorithm is due to the problem structure. For this particular scheduling problem, it is much easier to solve the MILP subproblem to a pre-defined optimality gap, e.g. 10%, than closing the gap to 0%.

Over all, the Newton's method seems to be the most efficient algorithm to solve this batch scheduling problem, because it is able to solve larger instances with more time points, leading to higher productivity than other algorithms/solvers. The inexact Newton's method with a proper gap (e.g. 10%) can use less time than exact Newton's method. We should also note that relative advantage of the inexact algorithm is somehow problem-dependent. We also test the inexact parametric algorithm in solving problem (TP1) and (TP2) presented in Section 3. The computational results of the random testing problems in Section 3 do not include those of the

Table 2

Model statistics and computational results of the simultaneous scheduling and controller selection problem (case study 2).

Inst. no.	Prod. no.	$D(i)$	Dis. Var.	Cont. Var.	Constr.	Obj. (\$/h)	CPU (s)	Solver
1	4	0.15	207	133	60	2542	2.219 (5 iter.)	Exact Newton's
						2542	2.203 (5 iter.)	Inexact Newton's (Gap 1%)
						2542	2.188 (5 iter.)	Inexact Newton's (Gap 10%)
						2542	3.016 (7 iter.)	Inexact Newton's (Gap 90%)
						2542	10.30(24 iter.)	Bisection
						2542	3.375 (7 iter.)	Secant
						2542	5.531 (11 iter.)	False position
						2542	3.3	BARON 11.3
						2542	0.266	SBB
						2559 ^a	0.141	DICOPT
2	8	0.15	959	645	212	2827	3.062 (4 iter.)	Exact Newton's
						2827	2.735 (4 iter.)	Inexact Newton's (Gap 1%)
						2827	2.547 (4 iter.)	Inexact Newton's (Gap 10%)
						2827	3.219 (5 iter.)	Inexact Newton's (Gap 90%)
						2827	13.38(24 iter.)	Bisection
						2827	4.672 (7 iter.)	Secant
						2827	7.687 (11 iter.)	False position
						2827	18.22	BARON 11.3
						2827	1.438	SBB
						2827	1.765	DICOPT
3	16	0.1	4095	4357	804	3309	58.52 (4 iter.)	Exact Newton's
						3309	28.94 (4 iter.)	Inexact Newton's (Gap 1%)
						3309	12.94 (4 iter.)	Inexact Newton's (Gap 10%)
						3309	22.61 (8 iter.)	Inexact Newton's (Gap 90%)
						3309	432.0 (24 iter.)	Bisection
						3309	140.9 (7 iter.)	Secant
						3309	195.6 (9 iter.)	False position
						3309	747.2	BARON 11.3
						3309	526.5	SBB
						3340 ^a	44.20	DICOPT
4	16	0.15	4095	4357	804	3395	80.77 (4 iter.)	Exact Newton's
						3395	65.17 (4 iter.)	Inexact Newton's (Gap 1%)
						3395	70.83 (4 iter.)	Inexact Newton's (Gap 10%)
						3395	42.70 (4 iter.)	Inexact Newton's (Gap 90%)
						3395	571.6 (23 iter.)	Bisection
						3395	92.14 (6 iter.)	Secant
						3395	202.0 (9 iter.)	False position
						3395	952.3	BARON 11.3
						3395	1002	SBB
						3395	51.38	DICOPT
5	16	0.2	4095	4357	804	3504	58.73 (4 iter.)	Exact Newton's
						3504	53.34 (4 iter.)	Inexact Newton's (Gap 1%)
						3504	65.45 (5 iter.)	Inexact Newton's (Gap 10%)
						3504	56.14 (5 iter.)	Inexact Newton's (Gap 90%)
						3504	442.9 (24 iter.)	Bisection
						3504	139.3 (7 iter.)	Secant
						3504	200.0 (9 iter.)	False position
						3504	618.9	BARON 11.3
						3504	643.2	SBB
						3504	50.66	DICOPT

^a Suboptimal solutions.

inexact methods, but the results are listed in the supporting information.

4.2. Simultaneous scheduling and controller design problems (mixed-integer quadratic fractional program)

The second example is an integrated scheduling and control problem for a continuous polymerization reactor that produces multiple products cyclically (Chu & You, 2012). As the focus of this paper is on the solution algorithms and their efficiency, details of the problem and model description are shown in the supporting information. We note that all the constraints of this problem are linear, and the objective function is the ratio of a convex quadratic function to a linear function. Thus, this is an MIQFP problem, and its corresponding parametric problem is a convex MIQP problem that can be solved efficiently with the branch-and-cut methods. We consider five instances with different numbers of products and demand

rate. We similarly solve these problems using the proposed parametric algorithms (MIQP solver CPLEX), MINLP solvers SBB, DICOPT and global optimizer BARON. The results are listed in Table 2.

As shown in Table 2, the problem size increases as the number of products increases. The parametric algorithms do not have obvious advantages over the general purpose MINLP solvers for solving the small scale problems, such as the first instance for the case of 4 products. However, for large problems with 16 products the parametric algorithms have clearly better performance than the general-purpose MINLP solvers. For the last three instances with 16 products, the parametric algorithms and the solvers SBB and BARON can obtain the global optimal solutions, but the parametric algorithms are at least 9 times faster than SBB and BARON. DICOPT can converge as fast as the proposed parametric algorithms but it might return a suboptimal local solution, as shown in the 1st and 3rd instances. Overall, the parametric algorithms work better than the general-purpose MINLP solvers in solving large

MIQFP problems in terms of both computational time and solution quality.

Among the proposed parametric algorithms, inexact Newton's method still has the best performance. For the instances that the inexact algorithms require the same numbers of iterations as the exact algorithm, the inexact approaches can converge to the global optimal solution much faster, because they do not need to solve each subproblem to 0% gap. The inexact algorithm with larger gap, such as 90%, however, may needs more iterations than the exact algorithm to converge to the same optimality tolerance, so it sometimes could end up with longer computational times than those inexact algorithms with smaller gaps and less iterations. Based on the comparison, it can be concluded that the inexact Newton's algorithms with appropriate gap could be much more efficient than the exact Newton's method in solving this problem. For certain instances, the computational time can be reduced by a factor of 4, such as the instance 3 that exact algorithm needs around 59 s while the inexact algorithm with 10% gap only needs 13 s.

5. Conclusion

In this paper, we presented parametric algorithms for solving the general MIFP problems. We first showed the equivalence between the parametric problem and the original MIFP problem, and then discussed the major properties of the equivalent parametric problem that can be used to facilitate the solution of large-scale MIFP problems. To solve the equivalent parametric problem we considered four one-dimension root-finding algorithms, including the bisection method, the Newton's method, the secant method and the false position method, respectively. These are all globally convergent exact algorithms. We further propose an inexact parametric algorithm, which is a variation of exact algorithm based on the Newton's method, for the global optimization of MIFPs problem. We conducted extensive computational studies based on the randomly generated instances, in order to test the efficiency of the four parametric algorithms for solving large-scale MILFP and MIQFP problems, and also to compare their performance with the general-purpose MINLP and global optimization methods, as well as a tailored MILFP solution method, the reformulation-linearization approach. The results showed that the proposed algorithms had higher computational efficiency than the general-purpose MINLP solvers. Through two example problems in process scheduling, the applicability and performance of the proposed algorithms were further illustrated and discussed. The results show that the exact and inexact parametric algorithms based on Newton's method can outperform other parametric algorithms, reformulation-linearization algorithm and the general-purpose MINLP algorithms in solving large-scale mixed integer fractional problems, and would lead to economic benefits when they are applied to process scheduling problems.

Appendix A. Properties of the parametric equivalent problem

Considering the following parametric problem (P_q):

$$F(q) = \max \{N(x) - q \cdot D(x) | x \in S\} \quad \text{for } q \in \mathbb{R}^1 \quad (P_q)$$

where x contains continuous variables and integer variables, S is nonempty, compact and bounded, $D(x) > 0$, and q is a parameter.

Below we show the special properties of this parametric problem (P_q).

Lemma 1 (convexity). $F(q) = \max \{N(x) - q \cdot D(x) | x \in S\}$ is convex over $\mathbb{R}^1$ (see You et al., 2009).

Lemma 2 (monotonically decreasing). $F(q) = \max \{N(x) - q \cdot D(x) | x \in S\}$ is strictly monotonically decreasing, i.e. $F(q'') < F(q')$ if $q' < q''$, $q', q'' \in \mathbb{R}^1$. (see You et al., 2009).

Lemma 3 (continuity). $F(q) = \max \{N(x) - q \cdot D(x) | x \in S\}$ is continuous for $q \in \mathbb{R}^1$

Proof. Let x^0 be the optimal solution of $F(q^0)$, then $D(x^0)$ is a positive constant and we also have $F(q^0) = \max \{N(x) - q^0 \cdot D(x) | x \in S\} = N(x^0) - q^0 \cdot D(x^0)$.

The function $F(q)$ is continuous, because it is monotonically decreasing (based on Lemma 2), and also for every $\varepsilon > 0$, we can find a $\delta = \varepsilon / D(x^0) > 0$, such that for all $q \in \mathbb{R}^1$, $0 \leq q - q^0 < \delta$ and $0 \leq F(q^0) - F(q) \leq \varepsilon$. The derivation is given below.

$$\begin{aligned} F(q^0) - F(q) &= \{N(x^0) - q^0 \cdot D(x^0)\} - \max \{N(x) - q \cdot D(x) | x \in S\} \\ &\leq \{N(x^0) - q^0 \cdot D(x^0)\} - \{N(x^0) - q \cdot D(x^0)\} \\ &= (q - q^0) \cdot D(x^0) \end{aligned}$$

Because $D(x^0) > 0$ and $0 \leq q - q^0 < \delta = \varepsilon / D(x^0)$, we have $F(q^0) - F(q) \leq \varepsilon$

□

Lemma 4 (bounded subgradients). $F(q) = \max \{N(x) - q \cdot D(x) | x \in S\}$ has bounded subgradients for $q \in \mathbb{R}^1$

Proof. Let x^0 be the optimal solution of $F(q^0)$ and let x^* be the optimal solution of $F(q^0 + h)$, then $D(x^0)$ is a positive constant and we have $F(q^0) = N(x^0) - q^0 \cdot D(x^0)$ and $F(q^0 + h) = N(x^*) - (q^0 + h) \cdot D(x^*)$.

For $h \rightarrow 0^+$ we have

$$\begin{aligned} \frac{F(q^0 + h) - F(q^0)}{h} &= \frac{\max \{N(x) - (q^0 + h) \cdot D(x) | x \in S\} - \{N(x^0) - q^0 \cdot D(x^0)\}}{h} \\ &\geq \frac{\{N(x^0) - (q^0 + h) \cdot D(x^0)\} - \{N(x^0) - q^0 \cdot D(x^0)\}}{h} \\ &= -D(x^0) \end{aligned}$$

$$\begin{aligned} \frac{F(q^0 + h) - F(q^0)}{h} &= \frac{\{N(x^*) - (q^0 + h) \cdot D(x^*)\} - \max \{N(x) - q^0 \cdot D(x) | x \in S\}}{h} \\ &\leq \frac{\{N(x^*) - (q^0 + h) \cdot D(x^*)\} - \{N(x^*) - q^0 \cdot D(x^*)\}}{h} \\ &= -D(x^*) \end{aligned}$$

$$\text{Thus, } -D(x^0) \leq \lim_{h \rightarrow 0^+} \frac{F(q^0 + h) - F(q^0)}{h} \leq -D(x^*)$$

Similarly, for $h \rightarrow 0^-$ we have $-D(x^0) \geq \lim_{h \rightarrow 0^-} \frac{F(q^0 + h) - F(q^0)}{h} \geq -D(x^*)$

Therefore, the subgradient for each point $q \in \mathbb{R}^1$ is bounded between $-D(x^0)$ and $-D(x^*)$.

When $h \rightarrow 0$, in most cases $x^* \rightarrow x^0$. Thus, $-D(x^0)$ can be used as an approximation of the derivative of $F(q)$.

Lemma 5 (unique solution). $F(q) = 0$ has a unique solution, say q^* (see You et al., 2009).

Appendix B. Algorithms for solving the equivalent parametric problem

B.1. Secant method

The secant method uses a succession of secant lines to approximate the root of the function. The secant method replaces the derivative in Newton's method with a finite difference to avoid the calculation of the derivative. However, this advantage of the secant method is not so significant for the parametric problem (P_q), because its derivative can be approximated by $-D(x^0)$ in Newton's method that is usually easy to calculate. Secant method converges

with an order of 1.6 approximately which is slower than Newton's method. Similar to the Newton's method, the convergence of secant method is not guaranteed in general root finding process; however, its convergence is guaranteed in this parametric problem (P_q), because $F(q)$ is strictly monotonically decreasing.

The procedure of the secant method for solving (P_q) is as follows:

Step 1. Choose arbitrary q' and q'' which satisfy $F(q') > 0$ and $F(q'') < 0$. Set $q_0 = q'$ and $q_1 = q''$; initialize n by setting $n = 2$.

Step 2. Let $q_n = q_{n-1} - F(q_{n-1}) \cdot \frac{q_{n-1} - q_{n-2}}{F(q_{n-1}) - F(q_{n-2})}$. Solve the problem $F(q_n) = \max \{N(x) - q_n \cdot D(x) | x \in S\}$. Denote the optimal solution as x_n^* .

Step 3. If $|F(q_n)| < \delta$ (optimality tolerance), stop and output x_n^* as the optimal solution and q_n as the optimal objective; if $|F(q_n)| \geq \delta$, update n with $n + 1$. Go to Step 2.

B.2. Bisection method

The simplest root-finding method is the bisection method, also called binary search method, which is motivated by the strictly decreasing property of the function $F(q)$ given in Lemma 2. Let q^* be the solution of the parametric problem, such that $F(q^*) = 0$. For a given value of q , if $F(q) > 0 = F(q^*)$, then the solution q^* must be larger than q , because the function is strictly decreasing. Thus, the range with the value less than q can be excluded from the search and the lower bound of the updated search range can be set as q . Similarly if $F(q) < 0 = F(q^*)$, then the range with the value larger than q can be excluded and the upper bound can be set as q . If the new value of q is selected as the middle point between the lower bound and the upper bound, the search range shrinks by half after each iteration step. Because the length of the search interval decreases to sufficiently small, the bisection method is guaranteed to converge to q^* , but it converges linearly.

The procedure of bisection method in solving (P_q) is as follows:

Step 1. Choose arbitrary q' and q'' which satisfy $F(q') > 0$ and $F(q'') < 0$. Set $q' = q'_0$ and $q'' = q''_0$; Initialize n by setting $n = 1$.

Step 2. Let $q_n = (q'_{n-1} + q''_{n-1})/2$. Solve the problem $F(q_n) = \max \{N(x) - q_n D(x) | x \in S\}$. Denote the optimal solution as x_n^* .

Step 3. If $|F(q_n)| < \delta$ (optimality tolerance), stop and output x_n^* as the optimal solution and q_n as optimal objective; if $F(q_n) \geq \delta > 0$, then let $q'_n = q_n$ and $q''_n = q''_{n-1}$, update n with $n + 1$; if $F(q_n) \leq -\delta < 0$, then let $q'_n = q_n$ and $q''_n = q'_{n-1}$, update n with $n + 1$. Go to Step 2.

B.3. False position method

The false position method or regula falsi method updates with the points, at which the line connecting the two points $(q'_n, F(q'_n))$ and $(q''_n, F(q''_n))$, intersects the q -axis as shown in Fig. 1. Similar to the bisection method, the false position method also requires two stating points q' and q'' with $F(q')$ and $F(q'')$ having opposite signs and the convergence rate is linear.

The procedure of the false position method for solving (P_q) is as follows:

Step 1. Choose q' and q'' which satisfy $F(q') > 0$ and $F(q'') < 0$. Set $q' = q'_0$ and $q'' = q''_0$ initialize n by setting $n = 1$.

Step 2. Let $q_n = \frac{F(q'_{n-1}) \cdot q''_{n-1} - F(q''_{n-1}) \cdot q'_{n-1}}{F(q'_{n-1}) - F(q''_{n-1})}$. Solve the problem $F(q_n) = \max \{N(x) - q_n D(x) | x \in S\}$. Denote the optimal solution as x_n^* .

Step 3. If $|F(q_n)| < \delta$ (optimality tolerance), stop and output x_n^* as the optimal solution. If $F(q_n) \geq \delta$, let $q'_n = q_n$ and $q''_n = q''_{n-1}$, update

n with $n + 1$. If $F(q_n) \leq -\delta$, let $q'_n = q_n$ and $q''_n = q'_{n-1}$, update n with $n + 1$. Go to Step 2.

B.4. The initialization of the root-finding algorithms

As shown above, the proposed algorithms require at least one initial value to start the root finding process. In this paper, one starting point q_0 is always chosen as 0. For bisection, secant, and false position methods in which two initial points are required, another starting point was determined by searching q'' and verifying the sign of $F(q'')$ with an exponential incremental step, such as 1, 10, 100, ..., until finding the $F(q'')$ with opposite sign of $F(0)$.

We note that the initialization approach might affect an algorithm's computational efficiency. In this work, however, the focus is on testing the MIFP theory and algorithm, so we use the same initialization scheme for all proposed algorithms for a fair comparison. Although it is beyond the scope of this work, it would be an interesting and important future extension of this work to study the roles of initialization and their impacts on the computational performance of the parametric algorithms and of the general-purpose MINLP methods.

Appendix C. Convergence rate of the exact Newton's method

In this appendix, we show that the exact parametric algorithm based on Newton's method has a superlinear convergence rate.

Lemma 6. Let x' and x'' be the optimal solution of $F(q')$ and $F(q'')$. If $q' < q''$, $D(x') \geq D(x'')$ (see You et al., 2009).

Lemma 7. Let x' and x^* be the optimal solution of $F(q')$ and $F(q^*)$, where q^* satisfies $F(q^*) = 0$. Then we have: $q^* - \frac{N(x')}{D(x')} \leq (q^* - q') (1 - \frac{D(x^*)}{D(x')})$ (see You et al., 2009).

Lemma 8. The exact parametric algorithm based on Newton's method has a superlinear convergence rate of $[1 - D(x^*)/D(x_k^*)]$, where x_k^* and x^* be the optimal solution of $F(q_k)$ and $F(q^*)$, and q^* satisfies $F(q^*) = 0$ (You et al., 2009).

Proof. Let q_k be the solution of iteration k , and q_{k+1} be the solution of the next iteration. Following the procedure of the exact Newton's method presented in Section 2.3, we have $q_{k+1} = N(x_k^*)/D(x_k^*)$.

Based on Lemma 7, we have: $q^* - q_{k+1} \leq (q^* - q_k)(1 - (D(x^*)/D(x_k^*)))$. Besides, when $k \rightarrow \infty$, $D(x_k^*) \rightarrow D(x^*)$.

Thus, we have $\lim_{k \rightarrow \infty} \frac{\|q_{k+1} - q^*\|}{\|q_k - q^*\|} = 0$. This implies that the algorithm has a superlinear convergence rate.

□

Appendix D. Convergence rate of the inexact parametric algorithm

This appendix shows the convergence rate of the proposed inexact parametric algorithm based on the Newton's method. Because this algorithm assumes the initial value of $q = 0$ and the numerator function $N(x) \geq 0$, we can easily prove Lemma 9 based on the strictly monotonically decreasing property of function $F(q)$.

Lemma 9. With the initial value of $q = 0$ and the numerator function $N(x) \geq 0$, we have $F(q) = \max \{N(x) - q D(x) | x \in S\} \geq 0$ for each subproblem of the algorithm.

Lemma 10. Let $\bar{x}'$ is within a relative optimality criterion of ε for $F(q')$, then $\eta \cdot [N(\bar{x}') - q' \cdot D(\bar{x}')] > F(q') = \max \{N(x) - q' \cdot D(x) | x \in S\}$, where $\eta = 1/(1 - \varepsilon)$.

Proof. Because we consider a similar definition of relative optimality criterion as used by GAMS, i.e. $(|BP - BF|)/|BP| < \varepsilon$, where BP

is the best possible solution, BF is the best solution found, and ε is the relative optimality criterion (Rosenthal, 2011). Because we only consider the cases that $F(q) \geq 0$, the absolute value signs of the formulation can be removed, i.e. $(BP - BF)/|BP| < \varepsilon$, and also $BP \geq 0$. Thus, we have $BP - BF < \varepsilon \cdot BP$, i.e. $BP < BF/(1 - \varepsilon) = \eta \cdot BF$.

In the specific problem of $\max \{N(x) - q' \cdot D(x) | x \in S\}$, for the feasible, near-optimal solution $\bar{x}$, $BF = N(\bar{x}) - q' \cdot D(\bar{x})$, while $BP = F(q') = \max \{N(x) - q' \cdot D(x) | x \in S\}$.

Thus, we have $\eta \cdot [N(\bar{x}) - q' \cdot D(\bar{x})] > F(q') = \max \{N(x) - q' \cdot D(x) | x \in S\}$

□

Lemma 11. Let x^* be the optimal solution of $F(q^*)$, where q^* satisfies $F(q^*) = 0$. Let $\bar{x}$ be a near-optimal solution of $F(q')$ with a relative optimality gap of ε . Then we have: $q^* - (N(\bar{x})/D(\bar{x})) < (q^* - q')(1 - (D(x^*)/\eta \cdot D(\bar{x})))$, where $\eta = 1/(1 - \varepsilon)$.

Proof. Because $\bar{x}$ is within a relative optimality criterion of ε for $F(q')$, and x^* is a feasible solution of $F(q')$, we can have the following inequality following Lemma 10:

$$\eta \cdot [N(\bar{x}) - q' \cdot D(\bar{x})] > N(x^*) - q' \cdot D(x^*)$$

Re-arranging both sides of the above inequality leads to the following one:

$$-N(x^*) + q' \cdot D(x^*) - \eta \cdot q' \cdot D(\bar{x}) > -\eta \cdot N(\bar{x})$$

Because $F(q^*) = N(x^*) - q^* \cdot D(x^*) = 0$, we have $N(x^*) = q^* \cdot D(x^*)$. Substituting this term to the above inequality, and also adding the term $\eta \cdot q^* \cdot D(x^*)$ to both sides, we have:

$$\eta \cdot q^* \cdot D(\bar{x}) - q^* \cdot D(x^*) + q' \cdot D(x^*) - \eta \cdot q' \cdot D(\bar{x}) > \eta \cdot q^* \cdot D(\bar{x}) - \eta \cdot N(\bar{x})$$

This inequality is equivalent to the one below:

$$(q^* - q') \cdot [\eta \cdot D(\bar{x}) - D(x^*)] > \eta \cdot D(\bar{x}) \cdot \left[q^* - \frac{N(\bar{x})}{D(\bar{x})} \right]$$

As $\eta \cdot D(\bar{x}) > 0$, dividing both sides by this term, we have:

$$q^* - \frac{N(\bar{x})}{D(\bar{x})} < (q^* - q') \cdot \left[1 - \frac{D(x^*)}{\eta \cdot D(\bar{x})} \right]$$

□

Remark 1. Lemma 11 shows that this inexact Newton's algorithm converges linearly with a rate of $[1 - D(x^*)/(\eta \cdot D(\bar{x}))]$.

Proof. Similar to the proof of Lemma 8.

Remark 2. The convergence rate is between the interval of (0, 1), i.e. $\eta \cdot D(\bar{x}) > D(x^*) > 0$.

Proof. Because $F(q^*) = \max \{N(x) - q^* \cdot D(x) | x \in S\} = N(x^*) - q^* \cdot D(x^*) = 0$, and $\bar{x}$ is a feasible solution of $F(q^*)$, we have:

$$N(x^*) - q^* \cdot D(x^*) = 0 \geq N(\bar{x}) - q^* \cdot D(\bar{x})$$

As $N(\bar{x}) - q^* \cdot D(\bar{x}) \leq 0$ and $\eta > 0$, it leads to the following inequality:

$$N(x^*) - q^* \cdot D(x^*) = 0 \geq \eta \cdot [N(\bar{x}) - q^* \cdot D(\bar{x})]$$

Because $\bar{x}$ is within a relative optimality criterion of ε for $F(q')$, and x^* is a feasible solution of $F(q')$, we have:

$$\eta \cdot [N(\bar{x}) - q' \cdot D(\bar{x})] > N(x^*) - q' \cdot D(x^*)$$

Adding the above two inequality together leads to the following valid inequality:

$$(q^* - q') \cdot [\eta \cdot D(\bar{x}) - D(x^*)] > 0$$

Based on the strictly monotonically decreasing property of function $F(q)$, we have $q^* > q'$ when the algorithm starts with the initial value of $q = 0$. Therefore, $\eta \cdot D(\bar{x}) > D(x^*) > 0$.

□

Appendix E. Supplementary data

Supplementary data associated with this article can be found, in the online version, at <http://dx.doi.org/10.1016/j.compchemeng.2013.10.017>.

References

- Anzai, Y. (1973). On integer fractional programming. *Journal of Operations Research Society of Japan*, 17, 49–65.
- Bank, B., Guddat, J., Klatte, D., Kummer, B., & Tammer, K. (1983). *Non-linear parametric optimization*. Stuttgart: Birkhäuser Verlag.
- Bazaraa, M. S., Sherali, H. D., & Shetty, C. M. (2006). *Nonlinear programming: Theory and algorithms*. Hoboken, NJ: John Wiley & Sons.
- Bradley, J. R., & Arntzen, B. C. (1999). The simultaneous planning of production, capacity, and inventory in seasonal demand environments. *Operations Research*, 47, 795–806.
- Bussieck, M. R., Dirkse, S. P., & Vigerske, S. (2007). *PAVER 2.0: An open source environment for automated performance analysis of benchmarking data*. GAMSWorld.
- Capón-García, E., Bojarski, A. D., Espuña, A., & Puigjaner, L. (2010). Multiobjective optimization of multiproduct batch plants scheduling under environmental and economic concerns. *AIChE Journal*, 57, 2766–2782.
- Chang, C. T. (2002). On the posynomial fractional programming problems. *European Journal of Operational Research*, 143(1), 42–52.
- Charnes, A., & Cooper, W. W. (1962). Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9, 181–186.
- Chu, Y., Wassick, J. M., & You, F. (2013). Efficient scheduling method of complex batch processes with general network structure via agent-based modeling. *AIChE Journal*, 59, 2884–2906.
- Chu, Y., & You, F. (2012). Integration of scheduling and control with online closed-loop implementation: Fast computational strategy and large-scale global optimization algorithm. *Computers & Chemical Engineering*, 47, 248–268.
- Chu, Y., & You, F. (2013). Integration of production scheduling and dynamic optimization for multi-product CSTRs: Generalized Benders decomposition coupled with global mixed-integer fractional programming. *Computers & Chemical Engineering*, 58, 315–333.
- Dinkelbach, W. (1967). On nonlinear fractional programming. *Management Science*, 13, 492–498.
- Dolan, E. D., & Moré, J. J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2), 201–213.
- Duran, M. A., & Grossmann, I. E. (1986). An outer-approximation algorithm for a class of mixed-integer nonlinear programs. *Mathematical Programming*, 36, 307–339.
- Floudas, C. A. (1999). *Deterministic global optimization: Theory, methods and applications*. Boston: Kluwer Academic Publishers.
- Frenk, H., & Schaible, S. (2009). Fractional programming. In C. A. Floudas, & M. Pardalos (Eds.), *Encyclopedia of optimization* (pp. 1080–1091). US: Springer.
- Gal, T. (1995). *Postoptimal analyses, parametric programming and related topics* (second ed.). Berlin: deGruyter.
- Geoffrion, A. M. (1972). Generalized benders decomposition. *Journal of Optimization Theory and Applications*, 10, 237–260.
- Granot, D., & Granot, F. (1977). On integer and mixed integer fractional programming problems. *Annals of discrete Mathematics*, 1, 221–231.
- Gupta, O. K., & Ravindran, A. (1985). Branch and bound experiments in convex nonlinear integer programming. *Management Science*, 31(12), 1533–1546.
- Hua, G., Cheng, T. C. E., & Wang, S. (2011). The maximum capture per unit cost location problem. *International Journal of Production Economics*, 131(2), 568–574.
- Isbell, J. R., & Marlow, W. H. (1956). Attrition games. *Naval Research Logistics Quarterly*, 3, 71–94.
- Ishii, H., Ibaraki, T., & Mine, H. (1977). Fractional Knapsack problems. *Mathematical Programming*, 13, 255–271.
- Jagannathan, R. (1966). On some properties of programming problems in parametric form pertaining to fractional programming. *Management Science*, 12, 609–615.
- Kesavan, P., Allgor, R. J., Gatzke, E. P., & Barton, P. I. (2004). Outer approximation algorithms for separable nonconvex mixed-integer nonlinear programs. *Mathematical Programming*, 100, 517–535.
- Kesavan, P., & Barton, P. I. (2000a). Generalized branch-and-cut framework for mixed-integer nonlinear optimization problems. *Computers & Chemical Engineering*, 24, 1361–1366.

- Kesavan, P., & Barton, P. I. (2000b). *Decomposition algorithms for nonconvex mixed-integer nonlinear programs*. New York: American Institute of Chemical Engineers. In AIChE Symposium Series.
- Li, H. L. (1994). A global approach for general 0–1 fractional programming. *European Journal of Operational Research*, 73(3), 590–596.
- Martos, B., & Andrew Whinston, V. (1964). Hyperbolic programming. *Naval Research Logistics Quarterly*, 11, 135–155.
- Mitsos, A., & Barton, P. I. (2009). Parametric mixed-integer 0–1 linear programming: The general case for a single parameter. *European Journal of Operational Research*, 194, 663–686.
- Murty, K. G. (1980). Computational-complexity of parametric linear-programming. *Mathematical Programming*, 19, 213–219.
- Pinto, J. M., & Grossmann, I. E. (1994). Optimal cyclic scheduling of multistage continuous multiproduct plants. *Computers & Chemical Engineering*, 18, 797–816.
- Pistikopoulos, E. N., Georgiadis, M. C., & Dua, V. (2007). *Multi-parametric programming: Theory, algorithms, and applications*. Germany: Wiley-VCH.
- Pochet, Y., & Warichet, F. (2008). A tighter continuous time formulation for the cyclic scheduling of a mixed plant. *Computers & Chemical Engineering*, 32(11), 2723–2744.
- Quesada, I., & Grossmann, I. E. (1992). An LP/NLP based branch and bound algorithm for convex MINLP optimization problems. *Computers & Chemical Engineering*, 16, 937–947.
- Ritter, K. (1962). Ein Verfahren zur Lösung parameterabhängiger, nichtlinearer Maximum- Probleme. *Unternehmensforschung. Band, 6(S.)*, 149–166.
- Rosenthal, R. E. (2011). *GAMS—A user's manual*. Washington, DC: GAMS Development Corp.
- Schaible, S. (1976). Duality in fractional programming—Unified approach. *Operations Research*, 24, 452–461.
- Schaible, S. (1981). Fractional programming: Applications and algorithms. *European Journal of Operational Research*, 7, 111–120.
- Schaible, S., & Ibaraki, T. (1983). Fractional programming. *European Journal of Operational Research*, 12, 325–338.
- Shah, N., Pantelides, C. C., & Sargent, R. W. H. (1993). Optimal periodic scheduling of multipurpose batch plants. *Annals of Operations Research*, 42(1), 193–228.
- Tawarmalani, M., & Sahinidis, N. V. (2005). A polyhedral branch-and-cut approach to global optimization. *Mathematical Programming*, 103, 225–249.
- Uddin, M. F., & Sano, K. (2010). Mixed integer linear fractional programming for integrated supply chain network design and optimization. *International Journal of Business and Economics*, 2(1), 55–70.
- Viswanathan, J., & Grossmann, I. E. (1990). A combined penalty-function and outer-approximation method for MINLP optimization. *Computers & Chemical Engineering*, 14, 769–782.
- Wassick, J. M., Agarwal, A., Akiya, N., Ferrio, J., Bury, S., & You, F. (2012). Addressing the operational challenges in the development, manufacture, and supply of advanced materials and performance products. *Computers & Chemical Engineering*, 47, 157–169.
- Westerlund, T., & Pettersson, F. (1995). A cutting plane method for solving convex MINLP problems. *Computers & Chemical Engineering*, 19, S131–S136.
- Wu, T. H. (1997). A note on a global approach for general 0–1 fractional programming. *European Journal of Operational Research*, 101(1), 220–223.
- You, F., Castro, P. M., & Grossmann, I. E. (2009). Dinkelbach's algorithm as an efficient method to solve a class of MINLP models for large-scale cyclic scheduling problems. *Computers & Chemical Engineering*, 33, 1879–1889.
- Yue, D., Kim, M., & You, F. (2013). Design of sustainable product systems and supply chains with life cycle optimization based on functional units: General Modeling Framework, MINLP Algorithms and Case Study on Hydrocarbon Biofuels. *ACS Sustainable Chemistry & Engineering*, 1, 1003–1014.
- Yue, D., Guillén-Gosálbez, G., & You, F. (2013). Global optimization of large-scale mixed-integer linear fractional programming problems: A reformulation-linearization method and process scheduling applications. *AIChE Journal*, 59, 4255–4272.
- Yue, D., & You, F. (2013). Sustainable scheduling of batch processes under economic and environmental criteria with MINLP models and algorithms. *Computers & Chemical Engineering*, 54, 44–59.